Difference Between Stack and Queue Data Structures

Last Updated : 23 May, 2024

In computer science, data structures are fundamental concepts that are crucial for organizing and storing data efficiently. Among the various data structures, **stacks** and **queues** are two of the most basic yet essential structures used in programming and algorithm design. Despite their simplicity, they form the backbone of many complex systems and applications. This article provides the differences between **stack** and **queue** data structures, exploring their characteristics, operations, and use cases.

Stacks

A stack is a linear data structure that follows the Last In, First Out (LIFO) principle. This means that the last element added to the stack is the first one to be removed. It can be visualized as a pile of plates where you can only add or remove the top plate.

Operations on Stack:

The primary operations associated with a stack are:

- **Push**: Adds an element to the top of the stack.
- **Pop**: Removes and returns the top element of the stack.
- **Peek (or Top)**: Returns the top element of the stack without removing it.
- **IsEmpty**: Checks if the stack is empty.
- **Size**: Returns the number of elements in the stack.

Use Cases of Stack:

Stacks are used in various applications, including:

- **Function Call Management**: The call stack in programming languages keeps track of function calls and returns.
- **Expression Evaluation**: Used in parsing expressions and evaluating postfix or prefix notations.

- **Backtracking**: Helps in algorithms that require exploring all possibilities, such as maze solving and depth-first search.

Queues

A **queue** is a linear data structure that follows the First In, First Out (FIFO) principle. This means that the first element added to the queue is the first one to be removed. It can be visualized as a line of people waiting for a service, where the first person in line is the first to be served.

Operations on Queue:

The primary operations associated with a queue are:

- **Enqueue**: Adds an element to the end (rear) of the queue.
- **Dequeue**: Removes and returns the front element of the queue.
- **Front (or Peek)**: Returns the front element of the queue without removing it.
- **IsEmpty**: Checks if the queue is empty.
- **Size**: Returns the number of elements in the queue.

Use Cases of Queue:

Queues are used in various applications, including:

- **Task Scheduling**: Operating systems use queues to manage tasks and processes.
- **Breadth-First Search (BFS)**: In graph traversal algorithms, queues help in exploring nodes level by level.
- **Buffering**: Used in situations where data is transferred asynchronously, such as IO buffers and print spooling.

Key Differences Between Stack and Queue

Here is a table that highlights the key differences between stack and queue data structures:

Feature	Stack	Queue
Definition	A linear data structure that follows the **Last In First Out (LIFO)** principle.	A linear data structure that follows the **First In First Out (FIFO)** principle.
Primary Operations	Push (add an item), Pop (remove an item), Peek (view the top item)	Enqueue (add an item), Dequeue (remove an item), Front (view the first item), Rear (view the last item)
Insertion/Removal	Elements are added and removed from the same end (the top).	Elements are added at the rear and removed from the front.
Use Cases	Function call management (call stack), expression evaluation and syntax parsing, undo mechanisms in text editors.	Scheduling processes in operating systems, managing requests in a printer queue, breadth-first search in graphs.
Examples	Browser history (back button), reversing a word.	Customer service lines, CPU task scheduling.
Real-World Analogy	A stack of plates: you add and remove plates from the top.	A queue at a ticket counter: the first person in line is the first to be served.
Complexity (Amortized)	**Push:** O(1), **Pop:** O(1), **Peek:** O(1)	**Enqueue:** O(1), **Dequeue:** O(1), **Front:** O(1), **Rear:** O(1)
Memory Structure	Typically uses a contiguous block of memory or linked list.	Typically uses a circular buffer or linked list.
Implementation	Can be implemented using arrays or linked lists.	Can be implemented using arrays, linked lists, or circular

Feature	Stack	Queue
		buffers.

Conclusion

Stacks and queues are fundamental data structures that serve different purposes based on their unique characteristics and operations. Stacks follow the LIFO principle and are used for backtracking, function call management, and expression evaluation. Queues follow the FIFO principle and are used for task scheduling, resource management, and breadth-first search algorithms. Understanding the differences between these two data structures helps in selecting the appropriate one for specific applications, leading to more efficient and effective programming solutions.

Next Article
Stack in C++ STL

Similar Reads

Stack Data Structure

A Stack is a linear data structure that follows a particular order in which the operations are performed. The order may be LIFO(Last In First Out) or FILO(First In Last Out). LIFO implies that the element that is...

3 min read

What is Stack Data Structure? A Complete Tutorial

Stack is a linear data structure that follows LIFO (Last In First Out) Principle, the last element inserted is the first to be popped out. It means both insertion and deletion operations happen at one end only....

4 min read

Applications, Advantages and Disadvantages of Stack

A stack is a linear data structure in which the insertion of a new element and removal of an existing element takes place at the same end represented as the top of the stack.Stack Data...

2 min read

Implement a stack using singly linked list

To implement a stack using a singly linked list, we need to ensure that all operations follow the LIFO (Last In, First Out) principle. This means that the most recently added element is always the first one to...

13 min read

Introduction to Monotonic Stack - Data Structure and Algorithm Tutorials

A monotonic stack is a special data structure used in algorithmic problem-solving. Monotonic Stack maintaining elements in either increasing or decreasing order. It is commonly used to efficiently solve...

12 min read

Difference Between Stack and Queue Data Structures

In computer science, data structures are fundamental concepts that are crucial for organizing and storing data efficiently. Among the various data structures, stacks and queues are two of the most basic yet...

4 min read

Stack implementation in different language

Some questions related to Stack implementation

Easy problems on Stack

Intermediate problems on Stack